LAB 4 (SECJ1023)

PROGRAMMING TECHNIQUE II

SECTION 03, 04 & 06, SEM 2, 2025/2026

1. Given the following program with two classes.

```
//Program 4.1
#include <iostream>
#include <iomanip>
#include <cstring>
using namespace std;

class kawan2;

class kawan1
{
      int value;
      char girlF[10];
      int age;
      void secret() {strcpy(girlF, "Amy"); age = 30;}
   public:
      kawan1(int a) { value = a; secret();}
      void print() { cout << " "<< value << " ";}
      friend void tukarGanti(kawan1 &, kawan2 &);
};

class kawan2
{
      int value;
      int age;
      char girlF[10];
      void secret() {strcpy(girlF, "Lisa");age=22;}
    public:
      kawan2(int a) { value = a; secret();}
      void print() { cout << " "<< value << " ";}
      friend void tukarGanti(kawan1 &, kawan2 &);
};

void tukarGanti(kawan1 &x, kawan2 &y)
{
      int temp = x.value;
      x.value = y.value;
      y.value = temp;
}

int main()
{
      kawan1 objek1(100);
      kawan2 objek2(50);
      cout << "\nValue before function tukarGanti()";
      objek1.print();
      objek2.print();
```

```
47        tukarGanti(objek1, objek2);
48        cout << "\nValue after function tukarGanti()";
49        objek1.print();
50        objek2.print();
51        return 0;
52  }
```

a. Type the Program 4.1. Study the code and run the program.

b. Change the definition of the above two classes with the following class definition in Program 4.2 and change the **`main()`** function to a new **`main()`**. Write **`gossip`** functions for **`kawan1`** and **`kawan2`** so that the output of the program is as shown in the Figure 4.1.

```
 1  //Program 4.2
 2  #include <iostream>
 3  #include <iomanip>
 4  #include <cstring>
 5  using namespace std;
 6
 7  class kawan2;
 8
 9  class kawan1
10  {
11        int value;
12        char girlF[10];
13        int age;
14        void secret() { strcpy(girlF, "Amy"); age = 30; }
15     public:
16        kawan1(int a) { value = a; secret(); }
17        void print() { cout << " "<< value << " "; }
18        void gossip (kawan2);
19        friend class kawan2;
20  };
21
22  class kawan2
23  {
24        int value;
25        int age;
26        char girlF[10];
27        void secret() { strcpy(girlF, "Lisa");age=22; }
28      public:
29        kawan2(int a) { value = a; secret();}
30        void print() { cout << " "<< value << " "; }
31        void gossip (kawan1);
32        friend void kawan1::gossip(kawan2);
33  };
34  // write the functions here
35
36  int main()
37  {
38        kawan1 objek1(100);
39        kawan2 objek2(50);
```

```
40        objek1.gossip(objek2);
41        objek2.gossip(objek1);
42        return 0;
43  }
```

```
100 gossips on 50
100 says 50 girl friend is Lisa
50 gossips on 100
50 says 100 age is 30
```

Figure 4.1

2. Given the following class definition of a **`Rectangle`**:

```
1  //Program 4.31
2  class Rectangle {
3      int width, height;
4    public:
5      Rectangle (int, int);
6      int calculateArea ();
7      friend Rectangle duplicate (Rectangle);
8  };
```

a. Write a meaningful implementation for the class **`Rectangle`**?

   i. The **`calculateArea`** function will return the area value of one **`Rectangle`** object.

   ii. The **`duplicate`** is a **`friend`** function. The function will receive one rectangle object as its argument. It will then multiply two of the argument's object width and height and return the object back to the caller.

b. Implement the class with the following **`main`** function in Program 4.4.

```
1  //Program 4.4
2  int main () {
3    Rectangle rect(10,30), rectb;
4    rectb = duplicate (rect);
5    cout << rectb.calculateArea();
6    system("pause");
7  }
```

c. Now, change the **`Rectangle`** class according to Program 4.5.

```
1  //Program 4.5
2  class Square;
3
4  class Rectangle {
5      int width, height;
6    public:
7      Rectangle(int, int);
8      int calculateArea ();
9      void convert (Square);
10 };
   class Square {
```

```
  private:
    int side;
  public:
    Square(int side){
                this->side=side;
    }
    friend class Rectangle;
};

```

3. Consider the **Length** class definition in the Program 4.6.

```
//Program 4.6
class Length
{
   private:
      int len_inches;
   public:
      Length (int feet, int inches)
      { len_inches=12*feet+inches;}
      int getFeet(){ return len_inches/12;}
      int getInches(){return len_inches%12;}
      friend bool operator<(Length , Length );
      friend bool operator==(Length a, Length b);
};
```

a. Write an overloaded **operator<** function using the given function definition. This function returns true if the first length is less than the second length.

b. Write an overloaded **operator==** function using the given function definition. This function returns true if both length is the same.

c. Write the main function which defines an array of object **Length** called **lens[]** with the size of 4. The function must follow the following rules:

   i. For each element of **lens**, set the values of **len_inches**.

   ii. Check which length are the same size using overloaded **operator==** and print message to screen if the compared length are the same. Example of the output if **lens[0]**, **lens[1]** and **lens[2]** are the same is as followed:

```
Same length at indexes 0 and 2
Same length at indexes 0 and 1
Same length at indexes 1 and 2
```

   iii. Check which length is the shortest and longest using overloaded **operator<** and print message to screen of the shortest length. Example of output is:

```
Shortest length in feet is 48 feet at index 0
Longest length in feet is 60 feet at index 3
```

4. Define a **Mechanic** class which has the following:

a. Member variables: name, staffid, hoursworked, salary, rate per hour.

b. Constructor that accepts the mechanic name, staff id, hours worked and rate per hour. These values should be assigned to the object's name, staff id, hours worked and rate per hour member variables

c. **`calculateSalary`**: This function will calculate salary of the mechanic based on the formula: **`salary = total hours worked * rate per hour`**.

d. **`display`**: This function will display all mechanic's information along with his salary.

e. **`calculateEPF`**: This is a **`friend`** function. This function will receive a **`Mechanic`** object as its argument. This function will calculate and return the **`epf`** for the object. The calculation for the **`epf`**: **`epf = 0.05 * salary`**.

Demonstrate the class in a **`main`** function that:

f. Creates a **`Mechanic`** object then,

g. Calculate the mechanic salary and display his information.

h. Display the mechanic's **`epf`** contribution value.

5. Design a class named **`Month`**. The class should have the following private member variables and public member functions:

a. **`name`** – of type string that holds the name of the month.

b. **`monthNumber`** – of type integer that holds the number of the month.

c. A constructor that accepts the name of the month as an argument. It should set **`name`** to the value passes as the argument and set **`monthNumber`** to the correct value.

d. Appropriate **`set`** and **`get`** functions for the **`name`** and **`monthNumber`** member variables.

e. **`++ prefix`** and **`postfix`** increment operators. These operators increment **`monthNumber`** and set **`name`** to the name of next month. If **`monthNumber`** is set to 12 when these functions execute, they should set **`monthNumber`** to 1 and **`name`** to "January".

f. You should also overload **`<< cout`**'s stream and **`>> cin`**'s stream operators to work with the **`Month`** class.

g. Demonstrate the class in a complete **`main`** function.

6. Given the following class definition:

```
//Program 4.7
class Product
{
 public:
   Product( )
   { prodCode=0; price=0.0; }
   int getProdCode(  )
   { return prodCode; }
   void setProdCode(int k)
   { prodCode=k; }
```

```
    double getPrice(  )
    { return price; }
    void setPrice( double p)
    { price=p; }
    friend ostream &operator<<( ostream&, const Product &);
  private:
    int prodCode;
    double price;
};
```

Write the definition of **friend** function **&operator<<**. For example, if object **ob1** has values **prodCode = 789** and **price = 12.50**, the statement **cout << ob1** should produce the following output:

```
Product code: 789
Product size: 12.50
```

7. Consider a class named **Date** representing the data of a date which consists of the following attributes: the **description** of the date which is of type string, **day**, **month**, and **year** which are of type integer. Write a C++ code by completing the source code in a file named **ssbt1.cpp**, according to the following tasks:

   i. Define the class **Date** which consists of
      a. all the attributes mentioned above.
      b. a constructor(s).
      c. an accessor for each attribute except the description.
      d. an overloaded minus (-) operator. This operator calculates the difference between two dates and returns the result in terms of a **Date** object. ***Note:*** The difference is calculated by subtracting each attribute (i.e., **day**, **month** and **year**), accordingly. All values must be positive. Below is an example of the calculation.

object	Day	month	year
date1	5	3	2018
date2	23	11	1999
date1 – date2	**18**	**8**	**19**

      e. an overloaded input (>>) operator to read the values for **day**, **month**, and **year** of the date from the keyboard. The header of the operator has been given. Complete the definition of the operator.
      f. an overloaded output (<<) operator to display the date in the following format. The header of the operator has been given. Complete the definition of the operator.

      Format: `description: day-month-year`

   ii. Complete the definition of function **displayDiff**. This function is meant to display the difference between two dates, in which the difference is given as a **Date** object.

The function at first checks the attribute **`year`**. If it has a non-zero value, the difference will be printed in terms of number of years. Otherwise, the function needs to check if the attribute **`month`** has a non-zero value. If so, the difference will be printed in terms of number of months, otherwise it will be in terms of number of days.

iii. Create an array to hold three (3) dates and initialize only their description with the following values: "SBT", "Test", and "Final", respectively.

iv. Using a loop and an appropriate operator, write the code that allows the user to enter the values of the day, month and year from the keyboard for each of the dates held in the array created in Task 3.

v. Create two other **`Date`** objects named **`date1`** and **`date2`**, with the following data, respectively:

Object	**`date1`**	**`date2`**
Description	TODAY	Date of birth
Day	5	23
Month	3	11
Year	2018	1999

vi. Using an appropriate operator and another loop, print the data of all the dates along with their difference to the object **`date1`**.

Figure 1 shows a sample screen output that your program should produce. Note that, **bold** texts indicate keyboard input entered by the user.

```
Enter your date (day, month, year): 11 3 2018
Enter your date (day, month, year): 30 3 2018
Enter your date (day, month, year): 5 6 2018
TODAY: 5-3-2018

Date of birth: 23-11-
1999 The difference: 19
year(s)

SBT: 11-3-2018
The difference: 6 day(s)

Test: 30-3-2018
The difference: 25 day(s)

Final: 5-6-2018
The difference: 3 month(s)
```

Figure 1: Example run of program

8. Define a class name Point based on the class diagram in **Figure 2**. The description for each function member is given below.

Point
- x: int - y: int
+ Point(int,int) + Point(const Point &) + display():void + display(string):void + operator+(Point):Point

Figure 2

Point (int,int)	Constructor to set the data members.
Point(const Point &)	Copy constructor.
display():void	Method that displays the coordinates. *Example:* x = 3 y = 6
display(string):void	Method that displays the coordinates with title. *Example A* Point 1 : x = 3 y = 6 *Example B* Coordinates : x = 5 y =10 **Note:** "Point 1 " and "Coordinates" are the titles of the points.
operator+(Point):Point	Overloaded + operator which is meant for point addition. *Example :* Point1: x = 1, y = 2. Point2: x = 2, y = 3. Point1 + Point2 : x = 3, y = 5.

Next, define a friend function to class Point name **slope** which calculates the slope between two points. The formula of calculating the slope is as follow:

$$slope = \frac{y_2 - y_1}{x_2 - x_1}$$

Finally, write an appropriate **main** function that do the following to produce the output shown in **Figure 3**:

a. Create two Point objects with the following coordinates: x=1, y=2 and x=3, y=4.
b. Create a third Point object and assign the object with the addition of two objects in (a).
c. Display the coordinates of the first and second points in (a) without title.
d. Display the coordinates of the third point in (b) with the title "Third point".
e. Print the slope between the two points in (a).

```
x= 1  y=2
x= 3  y=4
Third Point: x= 4     y=6
Slope = 1
```

Figure 3: Sample output

Table 1: Assessment Criteria

Item	Criteria	Marks
A	i) The program is able to run.	2
	ii) Using an appropriate structure for the program (e.g. the code is properly indented, all the required header files are included, the main function is properly written, etc.).	1
B	The definition of class Point	
	i) Data members	1
	ii) Point(int,int)	2
	iii) Point(const Point &)	3
	iv) display():void	2
	v) display(string):void	3
	vi) operator+(Point):Point	4
C	The declaration and definition of friend function **slope.**	3
D	Function **main**	
	i) Object 1 and object 2 creation	2
	ii) Object 3 creation	2
	iii) Coordinates display of the two objects without title.	2
	iv) Third coordinates display with title.	1
	v) Print the slope between two points	2
Total		**30**

9. A box can be represented by its dimensions, *width*, *length* and *height*. Then, the volume of the box is calculated by multiplying the dimensions, i.e., *volume* = *width* x *length* x *height*. Complete the source code file, **sbt1.cpp**, according to the following tasks:

   i. Define a class named **`Box`** which consists of
      a. all the required attributes representing the dimensions, (3 marks)
      b. constructors (or a constructor with default arguments), (4 marks)
      c. mutators, (6 marks)
      d. accessors, and (6 marks)
      e. a method that calculates the volume. (3 marks)

   ii. Complete the definition of function **`printBox`**. This function prints the dimensions and volume of a box. (4 marks)

   iii. Create an array to hold 5 boxes and initliize the first and second boxes with specified dimensions. (4 marks)

   iv. Using a loop, set the dimensions the remaining boxes with values entered from the keyboard. (7 marks)

   v. Using the function **`printBox`** and another loop, print all the boxes. (3 marks)

   vi. Calculate and print the total volume of all boxes. (5 marks)

Figure 4 shows what your program should look like when it runs. Note that, the **bold** texts indicate keyboard input entered by the user.

```
Enter box dimensions (w l h) => 2 3 4
Enter box dimensions (w l h) => 10 5 20
Enter box dimensions (w l h) => 3 3 3

Dimension = 10 x 10 x 10
Volume    = 1000

Dimension = 20 x 20 x 20
Volume    = 8000

Dimension = 2 x 3 x 4
Volume    = 24

Dimension = 10 x 5 x 20
Volume    = 1000

Dimension = 3 x 3 x 3
Volume    = 27

Total volume = 10051
```

Figure 4

10. Particle collision is the fundamental principle behind the invention of cathode ray tube (CRT) televisions (TVs) and computer monitors. The CRT takes particles, called electrons, from the cathode, speeds them up using electromagnets and then smashes them into phosphor particles on the screen. The collision between the electron and phosphor particles results in a lighted spot, or pixel on your TV or computer monitor. Develop a program to be used as a particle collision detector. This program determines whether two particles are colliding. For simplicity, particles are assumed to be spheres in shape. Thus, each particle can be represented by its center point and radius.

Your program should provide a class named ***Particle***, for representing a particle. The class has the following members:

a. Private data:
   i. The coordinates of center point, ***x***,***y*** and ***z***, respectively.
   ii. The radius, ***r***

b. Public operations:
   i. A default constructor.
   ii. A mutator function named ***read***. This function reads input from the keyboard to set the attributes ***x, y, z*** and *r*.
   iii. A function named **print** that displays all the attributes onto the screen.
   iv. An operator **−** that will be used for operations like:

   particle1 − particle2

   where ***particle1*** and ***particle2*** are of type ***Particle***. The above operation results in

the distance between those two particles. The distance between two particles is calculated by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

where (x_1, y_1, z_1) and (x_2, y_2, z_2) are the coordinates of the centre points for the two particles respectively.

Your program should also define a standalone or ordinary function named ***collision*** to test whether two particles are colliding. A collision is detected when the distance between two particles is less than or equal to the sum of their radius. The following figures illustrate this scenario.

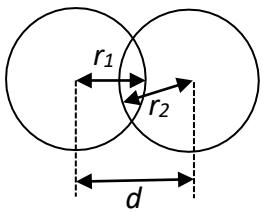

Figure 5: Collision

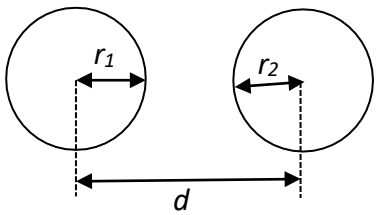

Figure 6: No collision

where ***r1*** and ***r2*** are the radius of two particles respectively, and ***d*** is the distance between the particles from their centers. A collision is detected in **Figure 5** as $d \leq (r_1 + r_2)$, while there is no collision in **Figure 6** as $d \geq (r_1 + r_2)$.

The function ***collision*** should receive two parameters of type ***Particle*** representing the two particles to be tested for collision. The function returns **true** if a collision is detected, otherwise **false**. Furthermore, specify the function to be a friend to the class ***Particle***. Also, the function should use the operator **–** from the class ***Particle*** to determine the distance between two particles.

Finally, utilize the class ***Particle*** and the function ***collision*** into the **main** function to realize the particle collision detector. Your program should look like as the example runs illustrated in **Figure 7**. The program should prompt an appropriate message whether a collision is detected or not. If a collision is detected, beside the prompt, the program should also display the information of the two particles.

As for the assessment criteria, refer to **Table 1**.

```
Enter data for the first particle:
Center: 0 0 0
Radius: 2

Enter data for the second particle:
Center: 2 2 2
Radius: 3

The particles are colliding.

First particle:
Center: (0,0,0)
Radius: 2

Second particle:
Center: (2,2,2)
Radius: 3
```

(a)

```
Enter data for the first particle:
Center: 0 0 0
Radius: 2

Enter data for the second
particle: Center: 10 20 30
Radius: 3

No collision detected.
```

(b)

Figure 7: Example runs of the program with (a) a collision is detected and (b) no collision detected. Note that the **bold** texts are user input.

Table 1: Assessment Criteria

No	Criteria	Marks
	Program structure:	
P1	The program is able to run	5
P2	Proper styles, *e.g.* indentation and comments have been applied	1
P3	Use an appropriate structure for the program (*e.g.* all required header files are included, the function main is properly written, *etc.*)	2
	Class declaration:	
C1	Declaration of class **Particle** (overall)	2
C2	Declaration all the member variables of the class	1
C3	The default constructor	2
C4	The method **read**	3
C5	The method **print**	2
C6	The **operator -**	3
	Standalone function:	
F1	Declaration of the function **collision**	1
F2	Specifying the function **collision** as friend to the class **Particle**	1
F3	Definition code for the function **collision** (overall)	1
F4	Code for the function **collision:** determining the distance	1
F5	Code for the function **collision:** determining collision	2
	The main function:	
M1	Creating two particles.	1
M2	Entering data for each particle.	2
M3	Collision testing	1
M4	Displaying a proper prompt whether a collision detected or not	2
M5	Displaying particles' information when a collision is detected.	2
	Total	**35**

11. A point can be described by its coordinates x and y. The distance between two points, $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$, is calculated by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

If the vertices (or points) of a polygon are $P_1, P_2, P_3, \ldots, P_n$ (as shown in **Figure 8**), the center of the polygon, *C,* can be determined by the following equation:

$$C = \frac{P_1 + P_2 + P_3 + \cdots + P_n}{n}$$

where *n* is the number of points.

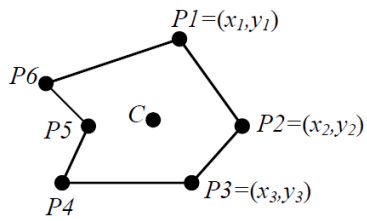

Figure 8: The representation of a polygon with six vertices (*i.e., n=6*).

Given the declaration of class Point in the file **sbtS5.cpp**. Without changing any code of the class declaration, edit the file to accomplish the following task requirements:

i) Complete the definition of the **overlaoded operator + (addition)** which adds up each coordinate of two points. For example, if the two points are *p1* = (1,2) and *p2* = (5,7), then *p1* + *p2* = (6, 9). (4 marks)

ii) Complete the definition of the **overlaoded operator – (subtraction)** which calculates the distance between two points. For example, if the two points are *p1* = (1,2) and *p2* = (5,7), then *p1* - *p2* = 6.403. (4 marks)

iii) Complete the definition of the **overlaoded operator / (division)** which divides each coordinate with the right operand. For example, if the point is *p* = (1,2), then *p* / 2.0 = (0.5, 1.0). (4 marks)

iv) Using the **overloaded input operator** and a proper loop, read a list of vertices from the keyboard. (3 marks)

v) Using another loop, calculate the center point by taking the average of the vertices of the polygon. (3 marks)

vi) Using the **overloaded output operator**, print the center point onto the screen. (1 mark)

vii) Using another loop, print the distance of each vertex to the center point of the polygon onto the screen. (5 marks)

Figure 9 shows example runs of the program.

```
Enter the number of vertices of a polygon => 3
Enter vertex 1 => 0 0
Enter vertex 2 => 2.5 0
Enter vertex 3 => 3.5 9

The center is (2.0,3.0)

Vertex   Coordinates     Distance to center
1        (0.0,0.0)       3.6
2        (2.5,0.0)       3.0
3        (3.5,9.0)       6.2
```

(a) A polygon with three vertices

```
Enter the number of vertices of a polygon => 5
Enter vertex 1 => 1 1
Enter vertex 2 => 2 2
Enter vertex 3 => 0 4
Enter vertex 4 => 1.5 3
Enter vertex 5 => 0 0


The center is (0.9,2.0)

Vertex   Coordinates     Distance to center
1        (1.0,1.0)       1.0
2        (2.0,2.0)       1.1
3        (0.0,4.0)       2.2
4        (1.5,3.0)       1.2
5        (0.0,0.0)       2.2
```

(b) A polygon with five vertices

Figure 9: Example runs of the program. User inputs are shown as **bold** texts.

12. Edit the source code file **sbt2.cpp** to accomplish the following task requirements:
Notes: You are not permitted to edit the code of the **main()** function.

    i) Complete the definition of default constructor. The constructor will create an array with the default size of 10 if the array size is not given when the object is created. The constructor also initializes each element of the array with the value 0. (5 marks)

    ii) Complete the definition of the **overloaded equality (==) operator** function. The function determines if two arrays are equal and return true, otherwise return false. The arrays are considered equal if their corresponding elements are equal. (7 marks)

    iii) Complete the definition of the **overloaded assignment (=) operator** function. The function copies all values from right array into left array, with assumption that two arrays have same size. (3 marks)

    iv) Complete the definition of the **overloaded addition (+) operator function**. The function returns an object of class `Array` that sums up the elements of left array with the passed object elements. Make sure the value of elements in the left array and the

passed object (right array) are not changed. (4.5 marks)

v) Complete the definition of the **overloaded input (>>) operator** function. The function input values for the entire array. (3 marks)

vi) Complete the definition of the **overloaded output (<<) operator**. The function displays two numbers per row of outputs. (7.5 marks)

Figure 10 shows the output of the program.

```
Size of array Num1: 4
After initialization, contents of array Num1:
   0  0
   0  0

Size of array Num2: 4
After initialization, contents of array Num2:
   0  0
   0  0

Enter 8 numbers:
1 2 3 4 5 6 7 8

After input, contents of array Num1:
   1  2
   3  4

After input, contents of array Num2:
   5  6
   7  8

Evaluating Num1 == Num2 >> Num1 and Num2 are not equal Assigning Num2
to Num1

After assigning, contents of array Num1:
   5  6
   7  8

After assigning, contents of array Num2:
   5  6
   7  8

Evaluating Num1 == Num2 >> Num1 and Num2 are equal Create new
object of class Array named Num3

After initializing, contents of array Num3:
   0  0
   0  0

After assigning new values, contents of array Num3:
   10 12
   14 16
```

Figure 10: Output of program